



INNOPOLIS
UNIVERSITY

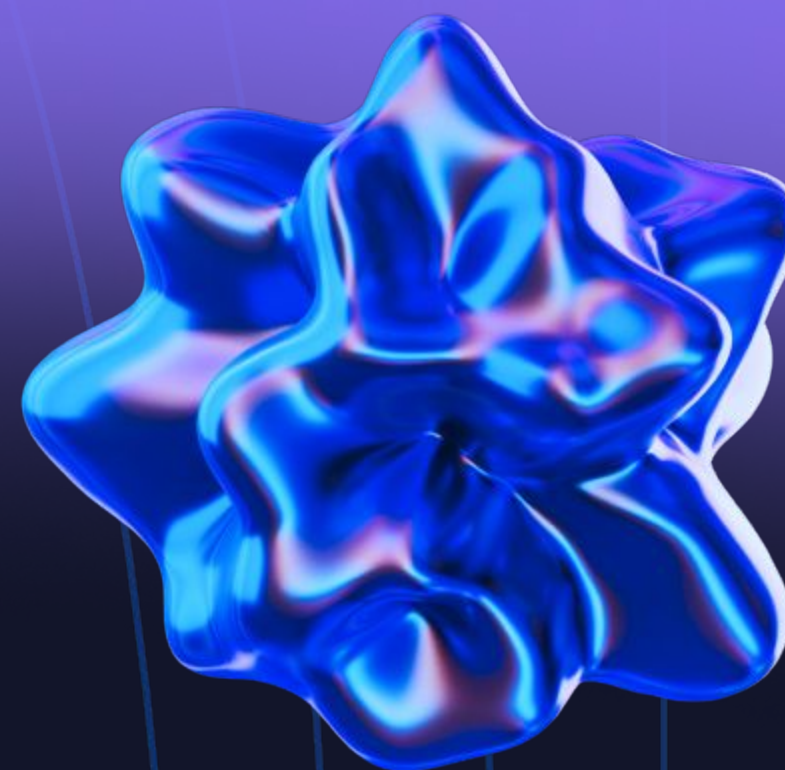
Student: Mikhail Voronin

Supervisor: Artem Burmyakov

Consultant: Alexander Tormasov

June 2026

Research and Adaptation of Components from the Persistent OS Phantom for Genode



Orthogonal persistence:

- Developed in the 1970s
- Not well-studied
- The hardware and network balances have changed
- Changes in requirements (e. g., power consumption)
- The current situation requires research

Why now?

It is necessary to examine the relevance of persistence concepts in the current environment

Objective

Different projects are implementing the concept
PhantomOS is our choice

Goal

Explore options for ensuring data security

Persistence is the period of time during which an object is usable¹

Two-level store (today's dominant model, non-persistent)

- Data is either active (RAM) or stored (disk)
- Programmer manually moves data between levels

Consequences

- Data loss
- I/O boilerplate ($\approx 30\%$ of code)
- Frequent initialization and configuration

¹ M. P. Atkinson, P. J. Bailey, K. J. Chisholm, W. P. Cockshott, R. Morrison. "PS-algol: A language for persistent programming", 1983.

Orthogonal Persistence

Alternative: the environment manages persistence

Principles¹

1. Independence
2. Data type orthogonality
3. Identification

Result

No two-level store drawbacks, and improved disk bandwidth utilization.

¹R. Morrison, M. P. Atkinson. “Persistent Languages and Architectures”, 1990.

PhantomOS¹ is a non-Unix-like OS implementing orthogonal persistence²

- Phantom Virtual Machine (PVM): bytecode VM of a persistent language
- Simplified POSIX layer (non-persistent)
- Periodical snapshots via **Copy-on-Write** (implementation of persistence)
- Changed pages written to disk (incremental)
- Programs temporarily suspended during snapshot preparation

¹<http://phantomos.org/>

²D. Zavalishin. PhantomOS, 2010.

PhantomOS ported to the Genode OS C++
framework¹

Why Genode

- Proven microkernels: NOVA, seL4, Fiasco.OC
- Capability-based security
- Minimal trusted computing base (TCB)

¹<https://genode.org/>

Snapper¹ is a file-system-based transactional snapshot mechanism²

Previous approach: “superblock”

- Snapshots stored as monolithic objects
- Fast, but no fault tolerance and no storage recovery

Snapper 2.0

- Configurable number of generations
- Unchanged pages linked, not copied
- ext4 + fsck for crash recovery
- Integrity checks (per-file hashes)
- Configurable redundancy policy

¹<https://github.com/rumenmitov/snapper>

²R. Mitov. “Snapper (v2): A PhantomOS snapshot mechanism”, FOSDEM 2026.

Problem Statement

Problem

- Snapshots contain **sensitive data**
- **Plaintext** snapshots are stored on the drive

Consequence

- Physical access attacks
 - Reproduce the machine state

Challenges

- **No prior research** on this for PhantomOS
- **No ready-made solution** exists in Genode

Solution

Implement a mechanism for the secure storage of snapshots.

No unauthorized access to data

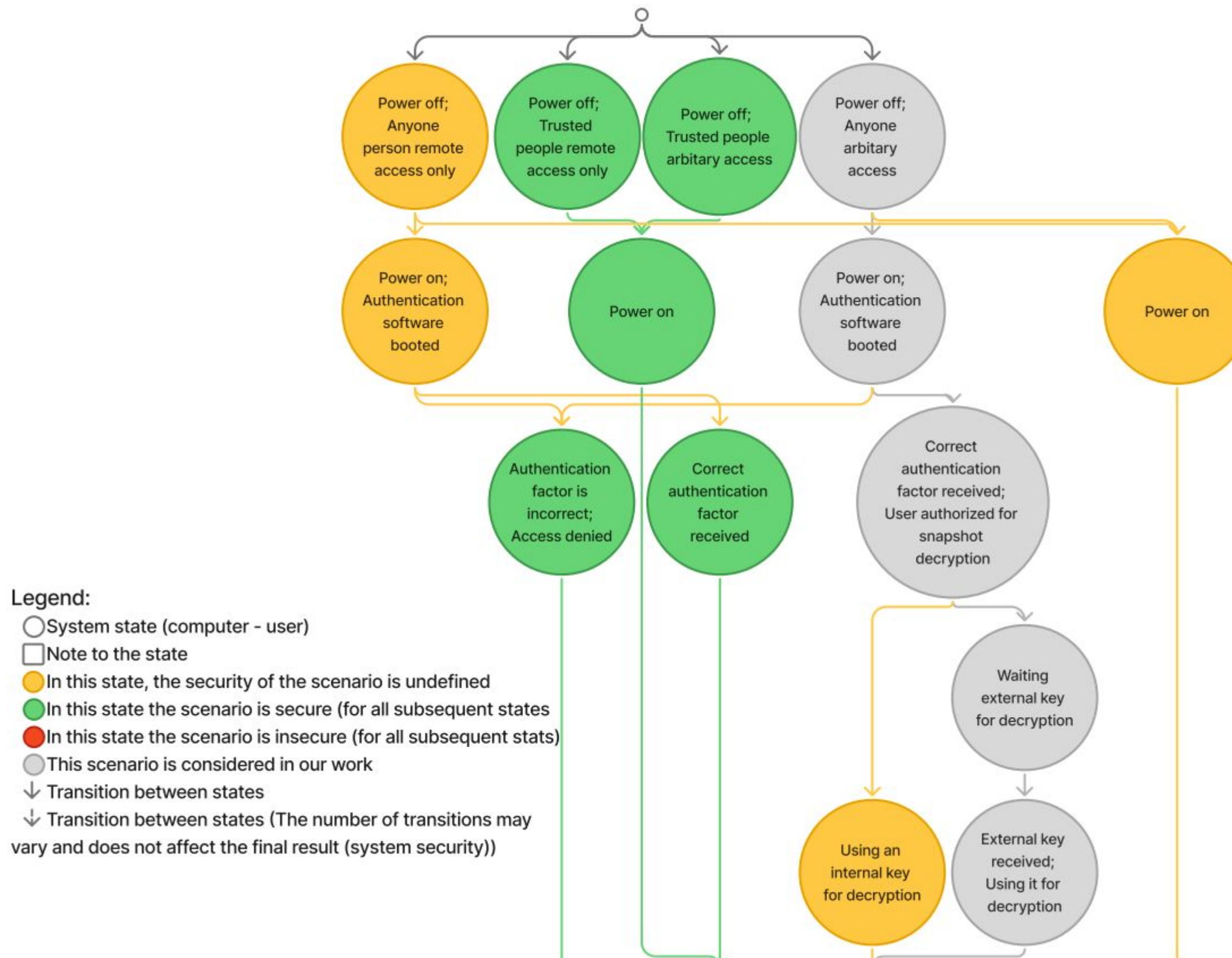
Requirements

1. Mandatory authorization
2. Key stored on an external USB drive
3. Encryption of snapshots (AES, block-level)
4. Only encrypted data written to disk
5. Acceptable performance overhead ($\leq 2\times$)

USB drive = key + auth factor

Requirements 1 and 2 combined: the USB drive both contains the key and serves as the authentication factor.

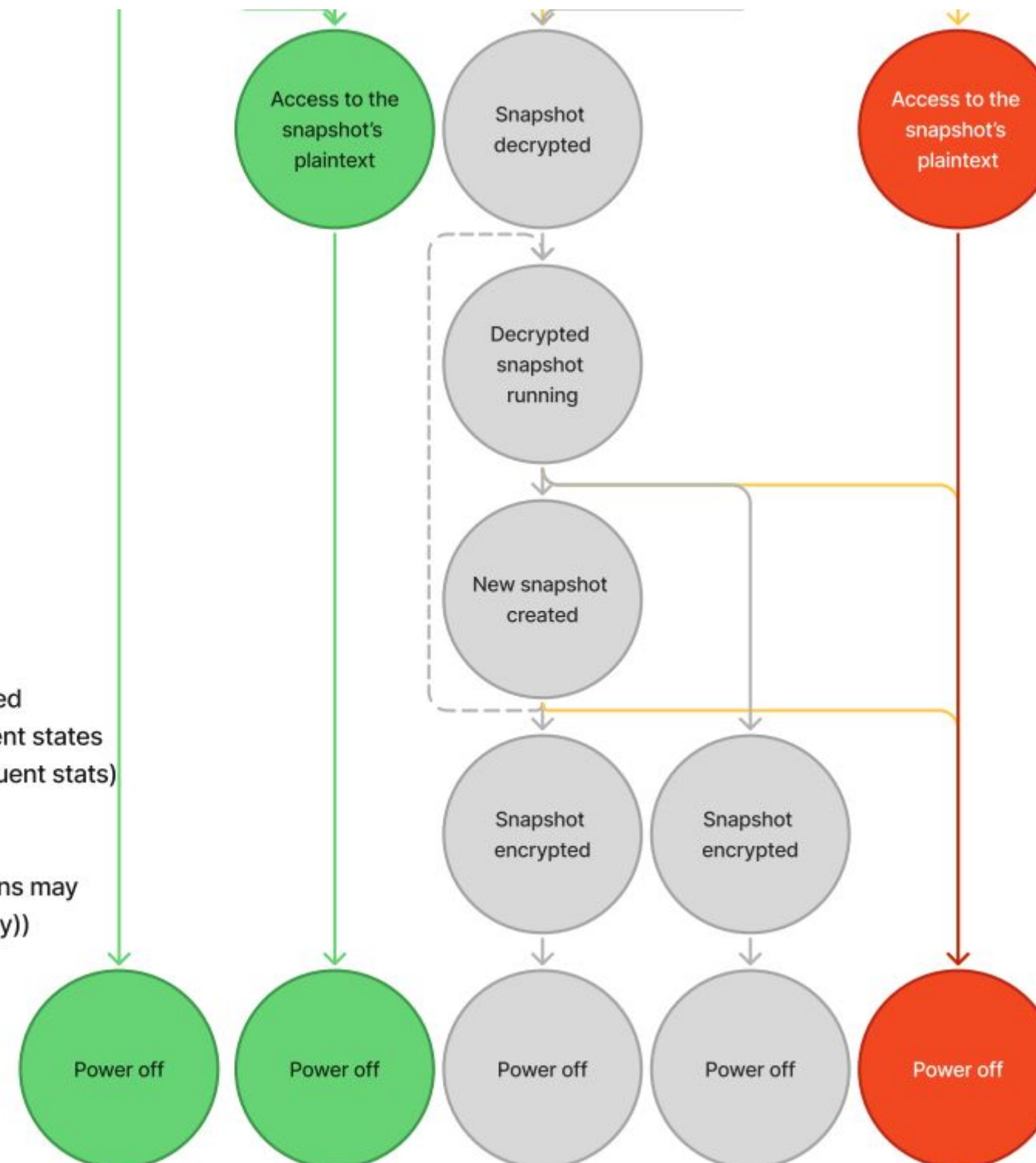
Usage Scenarios (1/2)



Usage Scenarios (2/2)

Legend:

- System state (computer - user)
- Note to the state
- In this state, the security of the scenario is undefined
- In this state the scenario is secure (for all subsequent states)
- In this state the scenario is insecure (for all subsequent states)
- This scenario is considered in our work
- ↓ Transition between states
- ↓ Transition between states (The number of transitions may vary and does not affect the final result (system security))



Implementation: se_vault

Built on file_vault² (based on Tresor)

- Tresor: AES on 4K blocks, SHA-256 integrity, CoW atomicity
- file_vault: state orchestrator providing a secure FS session

Implemented se_vault¹ as modified file_vault

Changes:

- Headless operation
- Passphrase replaced **by USB key file**
- rump_fs/ext2 replaced by **lwext4/ext4**
- Added **fsck** crash-recovery support
- Direct block session (skip image file)
- All block sizes standardized to 4K
- Graceful shutdown sequence added

²¹https://github.com/genodelabs/genode/tree/sculpt-25.04/repos/gems/src/app/file_vault

²https://github.com/VoronM1522/genode/tree/se/repos/gems/src/app/se_vault

Related Fixes & Adaptations

lwext4

ftruncate and sync

isomem

power-off button callback and graceful shutdown

snapper

client counter and graceful shutdown

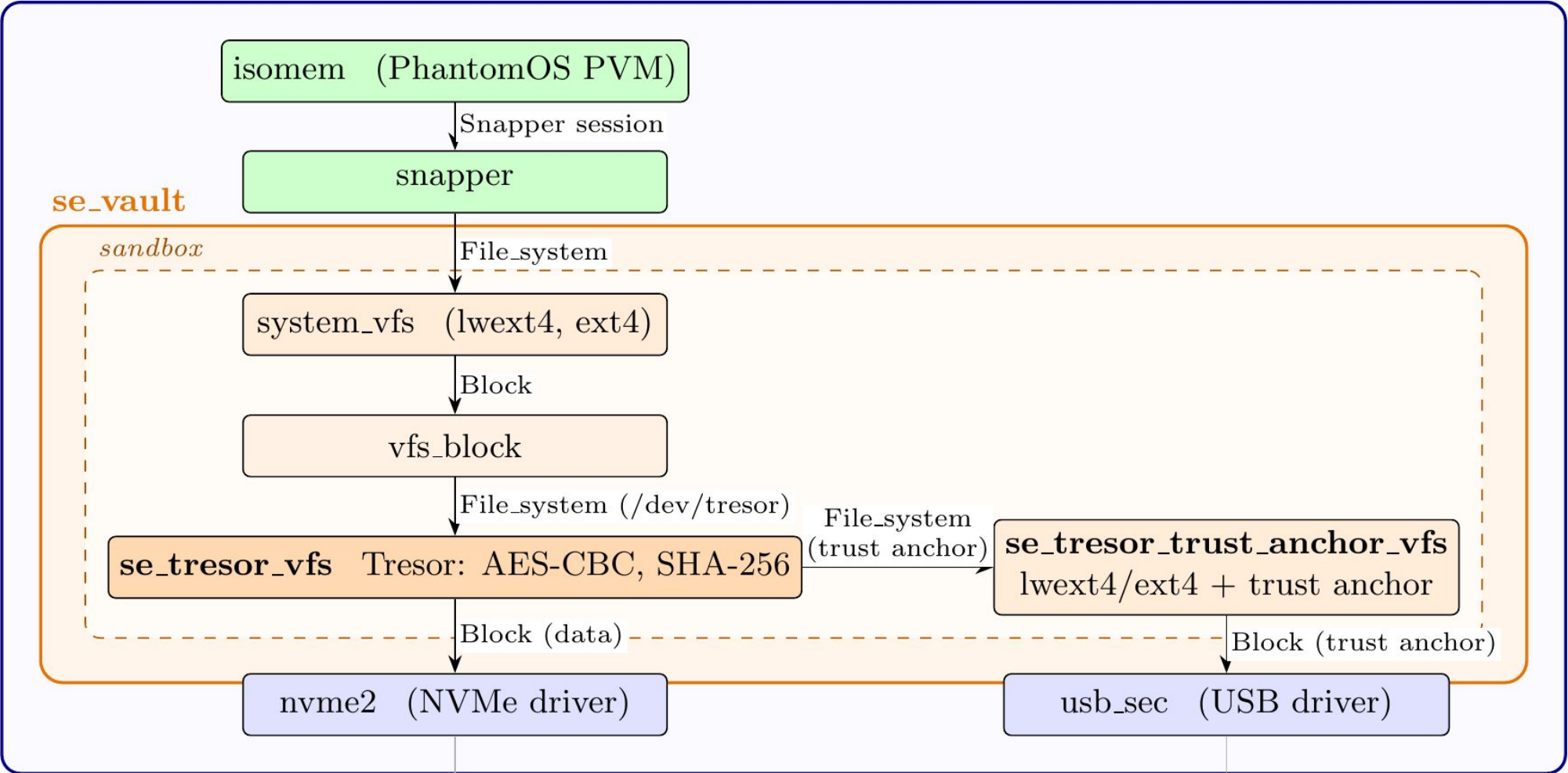
vfs

client counter, graceful shutdown and clean unmount

Optimization: using a “Block” session directly — initialization time reduced from ≈ 20 min to ≈ 1 min

System Architecture

Genode OS Framework



QEMU -cpu host, virtual NVMe (PCIe), USB passthrough

Podman container (USB flash passthrough: `--device /dev/bus/usb/...`)

Host Linux

- NVMe disk (btrfs): `block0.raw`, `test.5gb.raw`
- USB flash (ext4): trust anchor key, initialization flag

HW: NVMe SSD, USB 2.0 flash drive

First Run Configuration

The system was running on a Linux host inside a Podman container. Virtualized using QEMU.

Environment features

QEMU with -cpu host

AES/SHA hardware acceleration

NVMe controllers

virtual NVMe over PCIe

2 GB image

for isomem

5 GB image

for se_vault (4 GB user FS)

Functional tests — all passed

✓ Normal boot / shutdown / restore cycle

✓ Encrypted image cannot be mounted without key

✓ Key/hash replacement attack — superblock not detected

✓ Crash recovery via *fsck*

Performance Testing

Set up: first snapshot creation and reboot with restore, 5 runs each.

≈ 24×

snapshot creation overhead

≈ 11×

snapshot restore overhead

Root cause

Known Tresor limitation — no multithreading optimization.

Operation	Without enc.	With enc.	Overhead
Snapshot creation	2.10 s	51.00 s	≈ 24×
Snapshot restore	2.50 s	27.30 s	≈ 11×

Statistical validity has not been verified

4 of 5 requirements met; Consistent with the considered scenario

- ✓ Mandatory authorization (USB key)
- ✓ Key on external medium
- ✓ AES block-level encryption
- ✓ Only encrypted data on disk
- ✗ Overhead $\leq 2\times$ (Tresor limitation)

The current overheads may be acceptable in certain scenarios.

Conclusion

- **Usage scenarios were examined**
- **A security approach was selected**
- **Developed se_vault based on file_vault**

Improvements

- Security gain: snapshot encryption enabled
- Integrity gain: SHA-256 at block level (Tresor) and hash at FS level (Snapper)

Future work

- Decrease performance overhead; encrypt the isomem block device
- Merge Snapper and Tresor snapshot concepts; bring the PoC to industrial scale



INNOPOLIS
UNIVERSITY

Questions

